Nama : I Wayan Abraham Parulian Adrianus
NPM : 23081010153

Abstrak

Laporan ini merangkum tiga sketch ESP32 yang diberikan: kendali motor DC dasar melalui Serial, kendali Blynk melalui Virtual Pin V0, dan integrasi kendali Blynk dengan ArduinoOTA. Repositori publik iMCLab menyebut kit sebagai Internet-Based Motor Control Lab dan memuat contoh program Arduino serta Python; foto kit pada laporan ini berasal dari repositori tersebut, bukan dokumentasi eksperimen pengguna [1]. Ketiga sketch berhasil dikompilasi secara terpisah untuk ESP32 Dev Module dengan Arduino-ESP32 core 2.0.11. Kompilasi hanya membuktikan kode dapat dibangun; tidak membuktikan koneksi Blynk, keberhasilan OTA, atau kinerja motor di perangkat fisik.

Bagian Python pada laporan ini adalah rancangan kendali dari laptop menggunakan pySerial melalui USB-Serial, karena tidak ada berkas Python di antara tiga sumber yang diberikan. Rancangan ini bukan firmware MicroPython pada ESP32. Hasil pengujian fisik dan data pengukuran belum tersedia.

> Batas bukti: laporan membedakan analisis kode, hasil kompilasi, dan hasil uji perangkat. Tidak ada serial log, screenshot dashboard, video, atau pengukuran RPM yang disediakan; karena itu hasil motor nyata ditandai belum diuji.

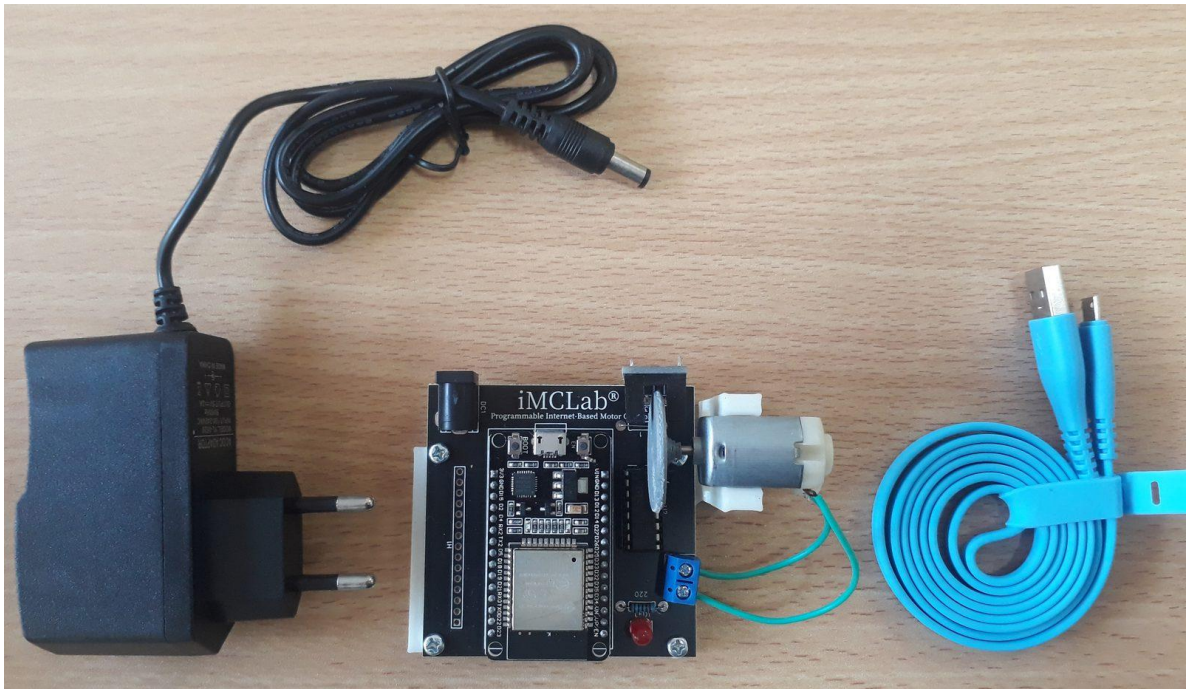
1. Tujuan dan ruang lingkup

Tujuan laporan adalah menjelaskan bagaimana motor DC pada Kit iMCLab dapat dikendalikan bertahap—dari perintah serial dasar, antarmuka Python pada komputer, kontrol jarak jauh melalui Blynk, hingga pembaruan firmware melalui OTA—serta menilai bukti dan batas implementasi dari tiga sketch sumber.

- Menguraikan pin, sinyal PWM, alur perintah, dan kondisi aman pada sketch dasar.
- Membedakan kontrol Python host melalui serial dari MicroPython yang berjalan di ESP32.
- Menganalisis pemetaan widget Blynk ke Virtual Pin V0 serta alur ArduinoOTA.
- Memisahkan hasil kompilasi yang diverifikasi dari hasil uji fisik yang belum dilakukan.

2. Gambaran kit dan metode riset

Repositori iMCLab memberikan konteks kit kontrol motor berbasis internet dan menampilkan contoh papan serta motor DC [1]. Pada tiga sketch pengguna, ESP32 memberi dua sinyal arah ke GPIO 27 dan 26, sinyal PWM ke GPIO 12, LED indikator di GPIO 2, dan pin input RPM di GPIO 13. Kode tidak membuktikan detail model driver, motor, catu daya, atau sensor encoder pada perangkat yang digunakan; detail itu harus dicatat saat praktikum.



Gambar 1. Contoh visual papan iMCLab, motor DC, adaptor, dan kabel USB pada repositori publik [1]. Foto ini hanya referensi visual, bukan foto perangkat atau hasil eksperimen pengguna.

Analisis gambar: papan bertanda iMCLab dan motor DC terlihat bersama catu daya serta kabel USB. Gambar membantu mengaitkan sketch dengan bentuk perangkat laboratorium, tetapi tidak dapat membuktikan pinout, tegangan, arus, atau hasil operasi motor. Verifikasi sambungan aktual tetap perlu dilakukan pada kit.

Metode dan bukti

Metode yang digunakan ialah pemeriksaan isi sketch, pencocokan handler dengan pin/datastream, pembuatan diagram konseptual, dan kompilasi tiap sketch di folder sementara yang terpisah. Pemisahan penting karena Arduino menggabungkan file .ino dalam satu sketch; tiga file yang masing-masing memiliki setup() dan loop() tidak boleh dikompilasi sebagai satu sketch gabungan.

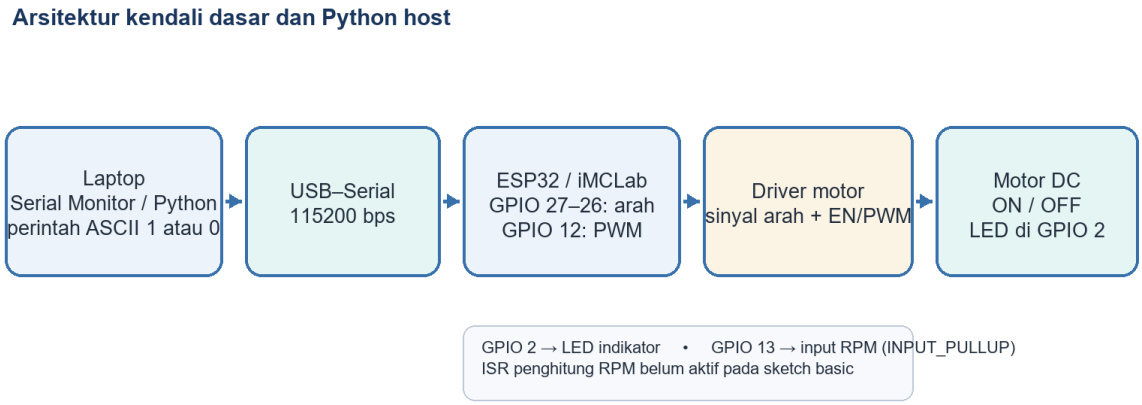
Jenis bukti	Status dalam laporan
Tiga sketch lokal	Dibaca dan dianalisis; sumber primer untuk perilaku kode.
Kompilasi ESP32	Lulus per sketch; ESP32 Dev Module, core 2.0.11.
Uji hardware / jaringan	Belum dilakukan; tidak ada log atau foto eksperimen pengguna.
Modul Python	Contoh host-side disusun dari antarmuka Serial pada sketch dasar; belum diuji ke perangkat.
Gambar	Satu foto referensi publik dan tiga diagram konseptual yang dibuat untuk laporan.

3. Laporan Riset Per Modul Eksperimen

3.1 Pemrograman basic robotika dengan Kit iMCLab

Sumber: on_off_Motor.ino. Tujuan modul adalah menguji kendali ON/OFF motor DC dan LED dari perintah karakter melalui Serial Monitor pada 115200 bps.

Bagian	Implementasi yang terbaca pada sketch
Arah motor	GPIO 27 = HIGH dan GPIO 26 = LOW untuk satu arah putar.
Enable / PWM	GPIO 12; frekuensi 30 kHz; resolusi 8 bit; channel 0 pada API ESP32 core 2.x.
Perintah	Karakter '1' memanggil MotorOn(); karakter '0' memanggil MotorOff().
Indikator	GPIO 2 diatur HIGH untuk ON dan LOW untuk OFF.
RPM	GPIO 13 INPUT_PULLUP; konfigurasi interrupt masih dikomentari dan rev tidak digunakan.



Gambar 2. Diagram konseptual alur Serial/komputer → ESP32 → driver → motor. Pemetaan pin diambil dari sketch dasar; detail driver aktual harus dicocokkan dengan kit.

Analisis gambar: perintah serial hanya membawa kode karakter; program menerjemahkannya menjadi level arah dan duty PWM. Driver motor berada di antara ESP32 dan motor karena motor DC tidak boleh ditenagai langsung oleh GPIO. Jalur input RPM digambarkan sebagai masukan, tetapi sketch belum mengaktifkan interrupt atau menghitung kecepatan.

Analisis PWM: dutyCycle = 205 pada skala 8-bit 0–255 setara sekitar 80,39% dari rentang duty (205/255 × 100). Nilai ini adalah perintah PWM, bukan hasil ukur kecepatan atau torsi; respons motor masih dipengaruhi tegangan catu, beban, driver, dan karakter motor. Fungsi MotorOff() menetapkan duty ke 0, tetapi mempertahankan pin arah HIGH/LOW. LEDC pada ESP32 menyediakan konfigurasi frekuensi dan resolusi untuk keluaran PWM [8].

Repositori publik memosisikan iMCLab sebagai kit kendali motor DC berbasis PID, tetapi sketch ini hanya menerapkan kendali open-loop ON/OFF dengan duty tetap; karena itu kode ini tidak membuktikan eksperimen PID [1]. Temuan dan keterbatasan: loop sketch dasar memanggil Serial.read() tanpa memeriksa Serial.available(), sehingga lebih baik diberi guard sebelum membaca. Setup juga belum memanggil MotorOff() untuk menetapkan keadaan awal secara eksplisit. Walaupun tersedia pin RPM, tidak ada data RPM, umpan balik, PID, atau pengukuran eksperimen dalam kode ini.

> Hasil fisik modul dasar: belum ada bukti uji. Berdasarkan logika kode, efek yang diharapkan adalah '1' mengaktifkan motor dan LED, sedangkan '0' memberikan duty 0 dan mematikan LED.

3.2 Pemrograman robotika dengan Kit iMCLab menggunakan Python

Tidak ada file .py di antara tiga sumber. Agar selaras dengan firmware dasar, modul ini dirumuskan sebagai program Python pada laptop yang mengirim byte '1' atau '0' melalui USB-Serial ke ESP32. Ini berbeda dari MicroPython: kode berikut berjalan di komputer dan membutuhkan pySerial, sementara firmware motor tetap sketch Arduino C++. pySerial menyediakan objek serial untuk membuka port dan menulis byte ke perangkat [7].

```
import time
import serial  # pip install pyserial

PORT = "COM5"  # ganti sesuai port ESP32
BAUD = 115200

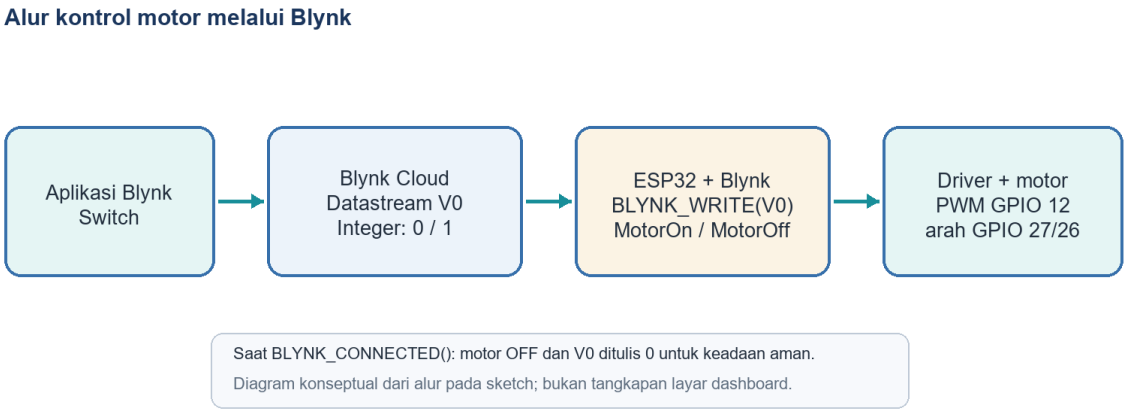
with serial.Serial(PORT, BAUD, timeout=1) as esp:
    time.sleep(2)  # beri waktu setelah port serial dibuka/reset
    print("Ketik 1 = motor ON, 0 = motor OFF, q = keluar")
    while True:
        command = input("> ").strip().lower()
        if command == "q":
            break
        if command in ("1", "0"):
            esp.write(command.encode("ascii"))
        else:
            print("Masukkan hanya 1, 0, atau q")
```

Analisis: program Python di atas memakai port COM dan baud rate yang sama dengan Serial.begin(115200) pada sketch dasar. Setelah port dibuka, program memberi jeda singkat agar papan sempat reset, lalu meneruskan karakter perintah yang memang dikenali firmware. Ganti COM5 sesuai port komputer dan tutup Serial Monitor sebelum menjalankan skrip karena satu port serial umumnya hanya dapat dibuka oleh satu aplikasi pada saat yang sama.

> Status modul Python: rancangan dan contoh kode, bukan hasil eksperimen. Belum dibuktikan pada Kit iMCLab pengguna. Bila yang dimaksud adalah MicroPython langsung di ESP32, firmware harus ditulis ulang untuk API GPIO/PWM MicroPython; contoh ini tidak melakukan port tersebut.

3.3 Pemrograman iMCLab Berbasis Blynk

Sumber: on_off_Motor_blynk_wifi.ino. Tujuan modul adalah mengendalikan status motor dari widget aplikasi melalui Wi-Fi dan Blynk Cloud. Dokumentasi Blynk menjelaskan penggunaan Virtual Pin untuk menghubungkan datastream/widget dan handler firmware; handler BLYNK_WRITE menerima perubahan nilai Virtual Pin [2][3].



Gambar 3. Diagram konseptual jalur kontrol aplikasi Blynk, datastream V0, ESP32, dan motor.

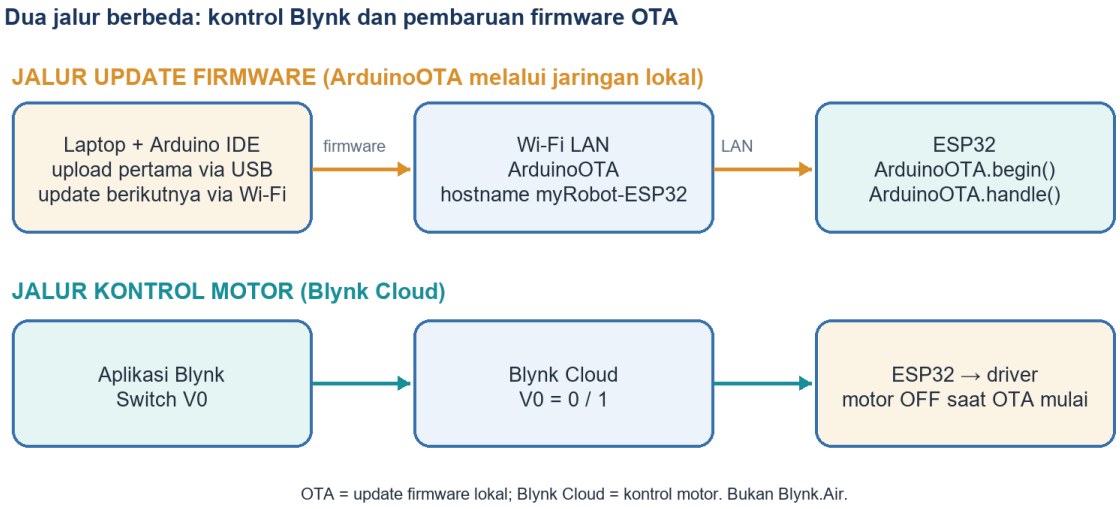
Analisis gambar: switch mengirim nilai V0 (0 atau 1) melalui Blynk Cloud; handler BLYNK_WRITE(V0) membaca param.asInt() lalu memilih MotorOff() atau MotorOn(). Pin arah dan duty PWM tetap dikendalikan lokal oleh ESP32. BLYNK_CONNECTED() memaksa motor OFF dan menulis V0 = 0 saat koneksi terbentuk kembali—perilaku fail-safe yang menghindari motor menyala hanya karena status lama aplikasi.

Pada implementasi ini, kontrol Blynk bersifat ON/OFF dengan duty tetap 205; belum ada slider kecepatan, telemetri RPM, atau pengendalian PID. Agar sesuai dengan firmware, dashboard perlu memakai datastream Virtual Pin V0 bertipe Integer dengan rentang 0–1. Kredensial Wi-Fi dan token sengaja tidak disalin ke laporan.

Uji yang direncanakan	Respons yang diharapkan dari kode	Status
Switch V0 = 1	MotorOn(); LED GPIO2 HIGH; PWM = 205.	Belum diuji fisik
Switch V0 = 0	MotorOff(); LED GPIO2 LOW; PWM = 0.	Belum diuji fisik
ESP32 tersambung ulang	Motor dipaksa OFF dan V0 ditulis 0.	Belum diuji fisik

3.4 Pemrograman OTA Berbasis Blynk

Sumber: on_off_Motor_OTA.ino. Sketch menggabungkan dua fungsi yang jalur komunikasinya berbeda: Blynk tetap dipakai untuk kontrol motor, sedangkan ArduinoOTA menyediakan penerimaan firmware melalui Wi-Fi. Pola ArduinoOTA yang lazim memerlukan inisialisasi layanan dan pemanggilan handler di loop; dokumentasi dan contoh resmi Espressif memperlihatkan pola OTA pada Arduino-ESP32 [4][5].



Gambar 4. Diagram konseptual yang memisahkan jalur update ArduinoOTA dari jalur kontrol motor Blynk.

Analisis gambar: untuk pemakaian pertama, firmware yang memuat dukungan OTA tetap perlu diunggah melalui USB. Setelah itu, komputer dan ESP32 berada pada jaringan lokal yang dapat saling menjangkau untuk upload ArduinoOTA. Pada kode, callback onStart memanggil MotorOff() agar motor tidak terus diberi PWM ketika proses flash dimulai; callback lain melaporkan progres dan jenis error.

Pembedaan penting: file ini bukan implementasi Blynk.Air. Blynk.Air adalah jalur OTA cloud yang terdokumentasi untuk alur firmware/Edgent Blynk; kode pengguna memakai ArduinoOTA lokal, sementara Blynk hanya tetap mengendalikan motor [6]. Karena itu, pembaruan jarak jauh melalui internet atau tombol di dashboard Blynk tidak dibuktikan oleh sketch ini.

Analisis keamanan dan reliabilitas: sketch OTA tidak mengatur kata sandi ArduinoOTA; sebaiknya tambahkan autentikasi OTA dan batasi akses hanya pada jaringan tepercaya. Selain itu, ArduinoOTA.begin() ditempatkan setelah Blynk.begin(); berarti layanan OTA baru didaftarkan setelah panggilan awal Blynk selesai. Perilaku ketika server Blynk tidak terjangkau belum diuji; untuk OTA yang tetap tersedia, koneksi Wi-Fi dan proses koneksi Blynk dapat dipisahkan/non-blocking.

> OTA yang diimplementasikan: pembaruan sketch melalui ArduinoOTA di jaringan lokal. Blynk pada sketch yang sama adalah kanal kontrol motor, bukan kanal transfer firmware.

4. Hasil kompilasi dan evaluasi

Kompilasi dilakukan memakai Arduino CLI 1.5.1 dan board ESP32 Dev Module (FQBN esp32:esp32:esp32), dengan platform ESP32 Arduino core 2.0.11. Masing-masing sketch disalin ke folder terpisah sebelum dikompilasi agar fungsi setup() dan loop() dari file lain tidak ikut digabung.

Sketch	Status compile	Flash terpakai	RAM global
on_off_Motor.ino	Lulus	268.025 byte (20%)	21.480 byte (6%)
on_off_Motor_blynk_wifi.ino	Lulus	736.837 byte (56%)	44.000 byte (13%)
on_off_Motor_OTA.ino	Lulus	775.649 byte (59%)	48.136 byte (14%)

Hasil kompilasi mengonfirmasi kompatibilitas sintaks dan library yang tersedia pada environment tersebut. Hasil ini tidak mengonfirmasi Wi-Fi tersambung, device Blynk online, motor berputar sesuai arah, nilai duty mencapai kecepatan tertentu, atau OTA berhasil mentransfer firmware; seluruhnya memerlukan uji perangkat dan jaringan.

Matriks uji lanjutan yang disarankan

Tahap	Prosedur singkat	Kriteria lulus	Status
Basic Serial	Kirim '1', lalu '0' dari Serial Monitor.	Motor/LED ON lalu OFF; motor tidak tersambung langsung ke GPIO.	Belum diuji
Python host	Jalankan skrip pySerial pada port ESP32.	Karakter 1/0 menghasilkan respons yang sama dengan Serial Monitor.	Belum diuji
Blynk	Ubah switch V0 dari 0 ke 1 dan kembali ke 0.	Perintah sampai ke ESP32 dan keluaran motor sesuai.	Belum diuji
Reconnect	Putus-sambung Wi-Fi atau restart ESP32.	Motor tetap OFF setelah koneksi ulang; V0 kembali 0.	Belum diuji
OTA	Mulai upload dari Arduino IDE saat motor awalnya ON.	onStart mematikan motor dan upload selesai tanpa reset manual tambahan.	Belum diuji

5. Diskusi dan simpulan

Ketiga firmware menunjukkan peningkatan jalur kendali: sketch dasar menerima perintah serial, sketch Blynk meneruskan ON/OFF melalui datastream V0, dan sketch OTA menambahkan pembaruan firmware lokal melalui ArduinoOTA. Perubahan ini meningkatkan kemudahan antarmuka dan pemeliharaan, tetapi belum mengubah sistem menjadi kendali kecepatan tertutup: sinyal RPM belum dihitung dan tidak ada loop PID.

Kesimpulan riset berbasis artefak: (1) logika dasar memetakan karakter 1/0 ke motor dan LED; (2) Blynk memakai V0 untuk ON/OFF dan menonaktifkan motor saat tersambung; (3) OTA memakai ArduinoOTA lokal dengan motor dimatikan pada awal update; (4) ketiga sketch lulus kompilasi pada ESP32 core 2.0.11; (5) modul Python merupakan rancangan host-side yang belum diuji; dan (6) belum tersedia bukti uji fisik, pengukuran RPM/kecepatan, atau keberhasilan koneksi cloud/OTA.

- Tambahkan keadaan motor OFF secara eksplisit pada setup sketch dasar, dan guard Serial.available() sebelum Serial.read().
- Aktifkan interrupt/encoder dengan debounce dan interval sampling jika target eksperimen adalah RPM atau kendali PID.
- Tambahkan autentikasi ArduinoOTA; simpan token dan kata sandi di file lokal yang tidak diunggah ke repositori.

- Catat tegangan catu, jenis driver, model motor, beban, duty PWM, dan RPM terukur agar laporan eksperimen berikutnya berbasis data.
- Simpan setiap sketch Arduino pada folder tersendiri sesuai nama file; jangan kompilasi tiga sketch independen sebagai satu folder.